Day 18 : V8 Engine (Memory Allocation)

Memory Ram architecture

Fixed Size data: Stack, dynamic size: Heap

4 bit architecture

Garbage Collector

Chart table

Unit	Symbol	Conversion	In Bytes
Byte	B	Base unit	1 B
Kilobyte	KB	1 KB = 1024 B	2^{10} B
Megabyte	MB	1 MB = 1024 KB	2^{20} B = 1,048,576 B
Gigabyte	GB	1 GB = 1024 MB	2^{30} B = 1,073,741,824 B
Terabyte	TB	1 TB = 1024 GB	2^{40} B ≈ 1.1 trillion B
Petabyte	PB	1 PB = 1024 TB	2^{50} B ≈ 1.13 quadrillion B
Exabyte	EB	1 EB = 1024 PB	2^{60} B ≈ 1.15 quintillion B
Zettabyte	ZB	1 ZB = 1024 EB	2^{70} B ≈ 1.18 sextillion B
Yottabyte	YB	1 YB = 1024 ZB	2^{80} B ≈ 1.21 septillion B

Where the Data is actually stored?

The Two Memory Regions in V8

When your JavaScript code runs, the V8 engine manages two primary areas of memory:

1. **The Call Stack:** A highly organized region for managing function execution. It's fast and stores **fixed-size** data. On a 32-bit system, the "slots" on the

stack are 32 bits (4 bytes) each.

2. **The Heap:** A large, less organized region for storing data that can change in size or has a longer lifetime. This is where objects, arrays, and other complex data live.

How Primitive Data is Stored (The V8 Reality)

This is where V8's optimizations are critical. It does **not** follow a simple "all primitives on the stack" rule.

number (The Hybrid Approach)

V8 splits numbers into two types to maximize performance:

1. **Smi (Small Integer):** code JavaScript
   - **What it is:** A 31-bit signed integer. This covers most numbers you use daily (from approx. -1 billion to +1 billion).
   - **How it's stored:** V8 uses a technique called **Pointer Tagging**. A 32-bit slot on the stack is used. The very last bit is a "tag." If this bit is 0, V8 knows the other 31 bits are an actual integer value.
   - **Where it's stored: Directly on the Stack.** A Smi is a true primitive in the classic sense. It is never allocated on the heap. This makes integer arithmetic extremely fast.

```
let loopCounter = 100; // This is a Smi. Its value lives on the stack.
```

2. **HeapNumber:** code JavaScript
   - **What it is:** Any number that is **not** a Smi. This includes all floating-point numbers (e.g., 19.5) and integers too large to be a Smi.
   - **How it's stored:** The value is "boxed" into a special object on the **Heap**. This object holds the full 64-bit (8-byte) representation of the number.
   - The variable on the **Stack** holds a 32-bit **pointer** (a memory address) to this object on the Heap. The tag bit for this pointer will be 1.

```
let price = 19.99; // This is a HeapNumber. 'price' on the stack is a pointer to an object on the heap.
```

string

- **How it's stored:** The actual character data of a string is **always on the Heap**. A string can be any length, so it's not suitable for the fixed-size stack.
- **Where it's stored:** The variable on the **Stack** holds a 32-bit **pointer** to the string object on the Heap.

boolean, null, undefined (The Singleton Objects)

- **How it's stored:** V8 creates **one, single, permanent object for true**, one for false, one for null, and one for undefined when the engine starts. These are called "Oddball" singletons and they live in a read-only part of the **Heap**.
- **Where it's stored:** When you declare let isActive = true;, the isActive variable on the **Stack** simply holds a 32-bit **pointer** to that one global true object on the Heap. This is highly efficient for memory and comparisons.

symbol and bigint

- **How it's stored:** These are complex types and are always allocated as objects on the **Heap**.
- **Where it's stored:** The variable on the **Stack** holds a 32-bit **pointer** to their object representation on the Heap.

How Non-Primitive Data (Objects) are Stored

This is simpler and more consistent.

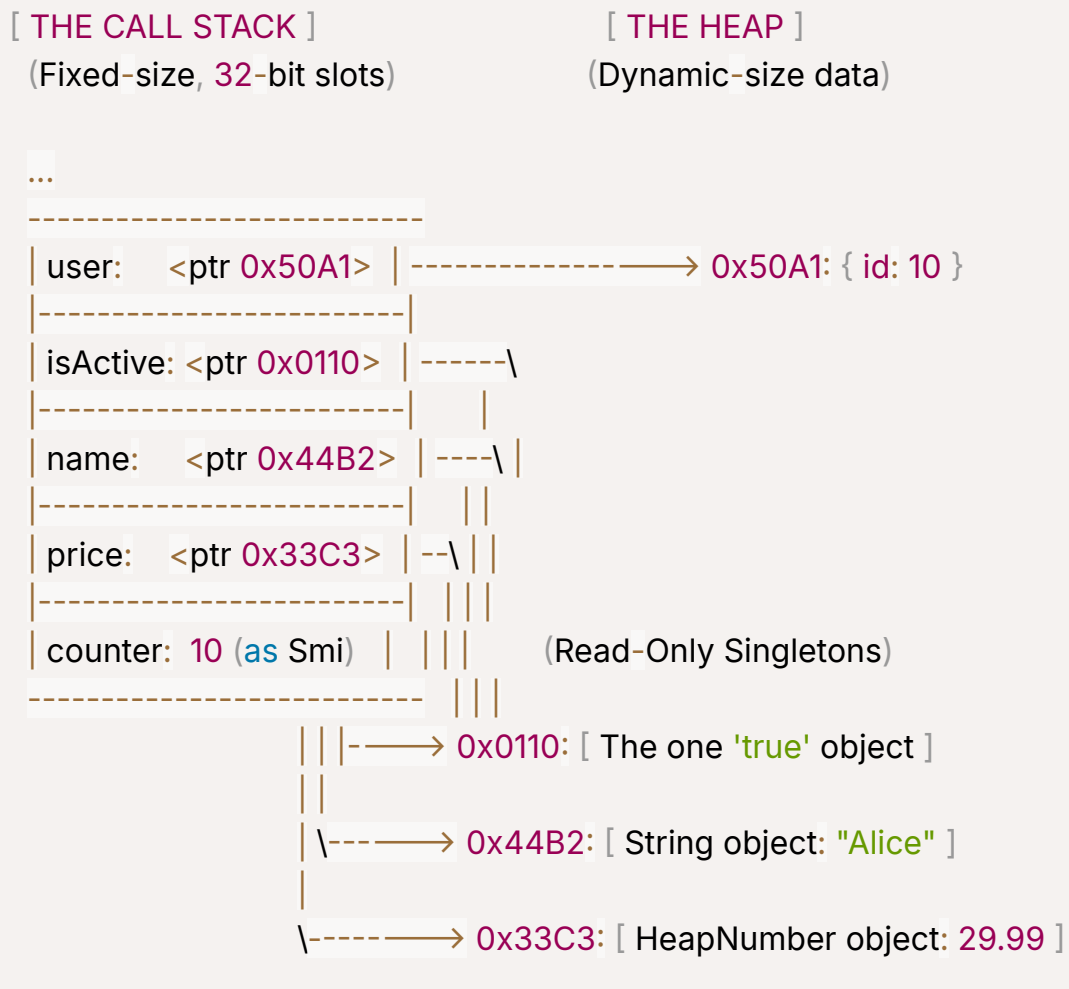
- **How it's stored:** An object (or array) and all of its properties and values are allocated on the **Heap**. This is because objects are dynamic; their size can change as you add or remove properties.
- **Where it's stored:** The variable you declare on the **Stack always** holds a 32-bit **pointer** to the object's location on the Heap.

Visual Summary: The Complete Picture

Let's trace this code on a 32-bit system:

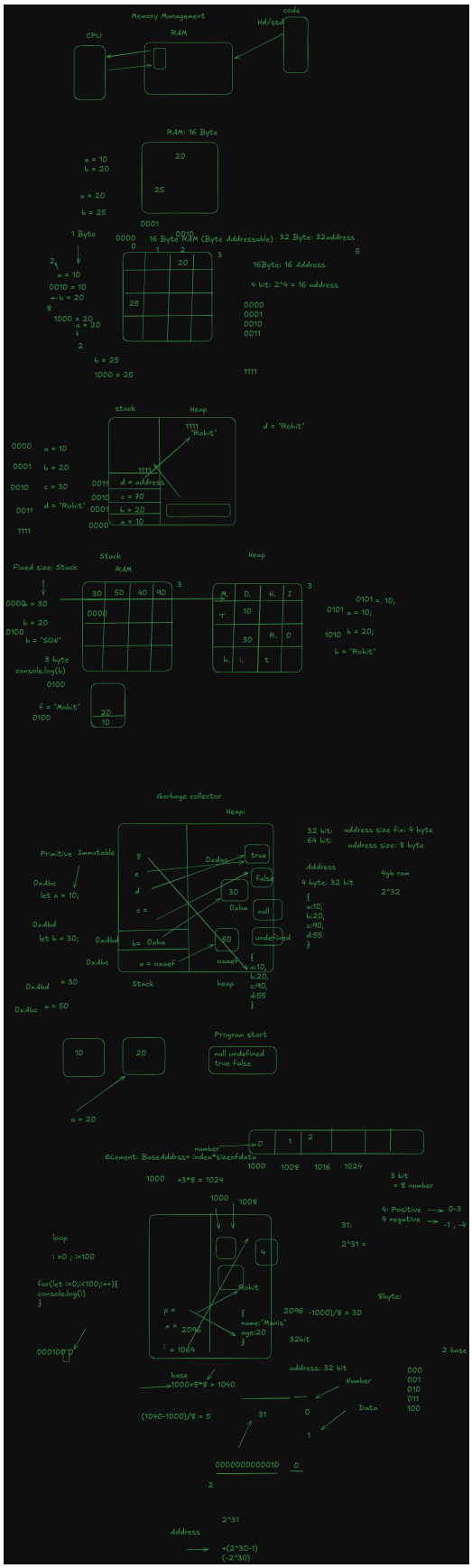
```
let counter = 10;            // Smi number
let price = 29.99;           // HeapNumber
let name = "Alice";             // String
let isActive = true;         // Boolean
let user = { id: counter };     // Object
```

Memory Layout:



Final Conclusion

Data Type	Stack or Heap? (32-bit V8)
number (small integer)	**Stack** (as a Smi)
number (float/large int)	**Heap** (variable on Stack holds a pointer)
string	**Heap** (variable on Stack holds a pointer)
boolean	**Heap** (variable on Stack holds a pointer to a singleton)
null	**Heap** (variable on Stack holds a pointer to a singleton)
undefined	**Heap** (variable on Stack holds a pointer to a singleton)
symbol / bigint	**Heap** (variable on Stack holds a pointer)
object / array	**Heap** (variable on Stack holds a pointer)


Memory Management
CPU
RAM
code
Hd/ssd
RAM: 16 Byte
a = 10
b = 20
a = 20
b = 25
1 Byte
16 Byte RAM (Byte Addressable)
32 Byte: 32address
16Byte: 16 Address
4 bit: 2^4 = 16 address
0000
0001
0010
0011
1111
b = 25
1000 = 25
stack
Heap
d = "Rohit"
a = 10
b = 20
c = 30
d = "Rohit"
c = 70
d = address
Stack
Heap
Fixed size: Stack
RAM
b = 20
b = "SOW"
3 byte
console.log(b)
f = 'Mohit'
0101 a = 10;
b = 20;
b = "Rohit"
Garbage collector
Heap:
Primitive
Immutable
let a = 10;
let b = 30;
a = 50
Stack
heap
32 bit:
64 bit:
address size Fix: 4 byte
address size: 8 byte
Address
4 byte: 32 bit
4gb ram
2^32
Program start
null undefined
true false
a = 20
number
Element: BaseAddress+ index*sizeofdata
1000
1008
1016
1024
1000 +3*8 = 1024
3 bit
= 8 number
4: Positive
4 negative
-1 , -4
31:
2^31 =
loop
i =0 ; i=100
for(let i=0;i<100;i++){
console.log(i)
}
Rohit
8byte:
p =
x = 2096
i = 1064
2096 -1000)/8 = 30
{
name:'Mohit'
age:20
}
32bit
2 base
address: 32 bit
base
=1000+5*8 = 1040
Number
Data
000
001
010
011
100
(1040-1000)/8 = 5
31
0
1
0000000000010
0
2
2^31
Address
+(2^30-1)
(-2^30)